Adv It increases customer satisfaction
It reduces errors & defects
It improves quality
It helps in better decision making using data.

Unit - V

(1) case & its scope [computer aided s/w Engineering] refers to the use of s/w tools to automate & support the activities involved in s/w dev^t & maintenance. Case tools help dev^{rs} do their work faster, easier & better quality. A case tool is any s/w that helps in performing tasks such as analysis, designing, coding, testing & maintenance. case tools are used in phase related activities such as Req specification, sys analysis, Design, coding & testing, non phase activities such as project mngmt, Docu^n, Configuration mngmt

Scope of case tools covers all stages of the s/w dev^t life cycle, helping dev^{rs} work efficiently & systematically. The work is an organized way. It saves time. It reduce errors & maintain s/w easily.

objectives of CASE tools

→ To increase productivity.
→ To produce high qua s/w

→ To reduce devt & maintenance cost

(2) CASE Environment is an integrated collection of case tools that work together to support the entire s/w devt life cycle. Instead of using separate tools, a case environment connects all tools in one sys, so they can share data & work together. When tools are not integrated data must be manually tranferred btn them. This causes extra effort, data conversion issues. So, case env^t solves this by integrating all tools into one framework. It uses a Central repository [data dic] where all project data is stored, all tools access the same info & consistency is maintained.

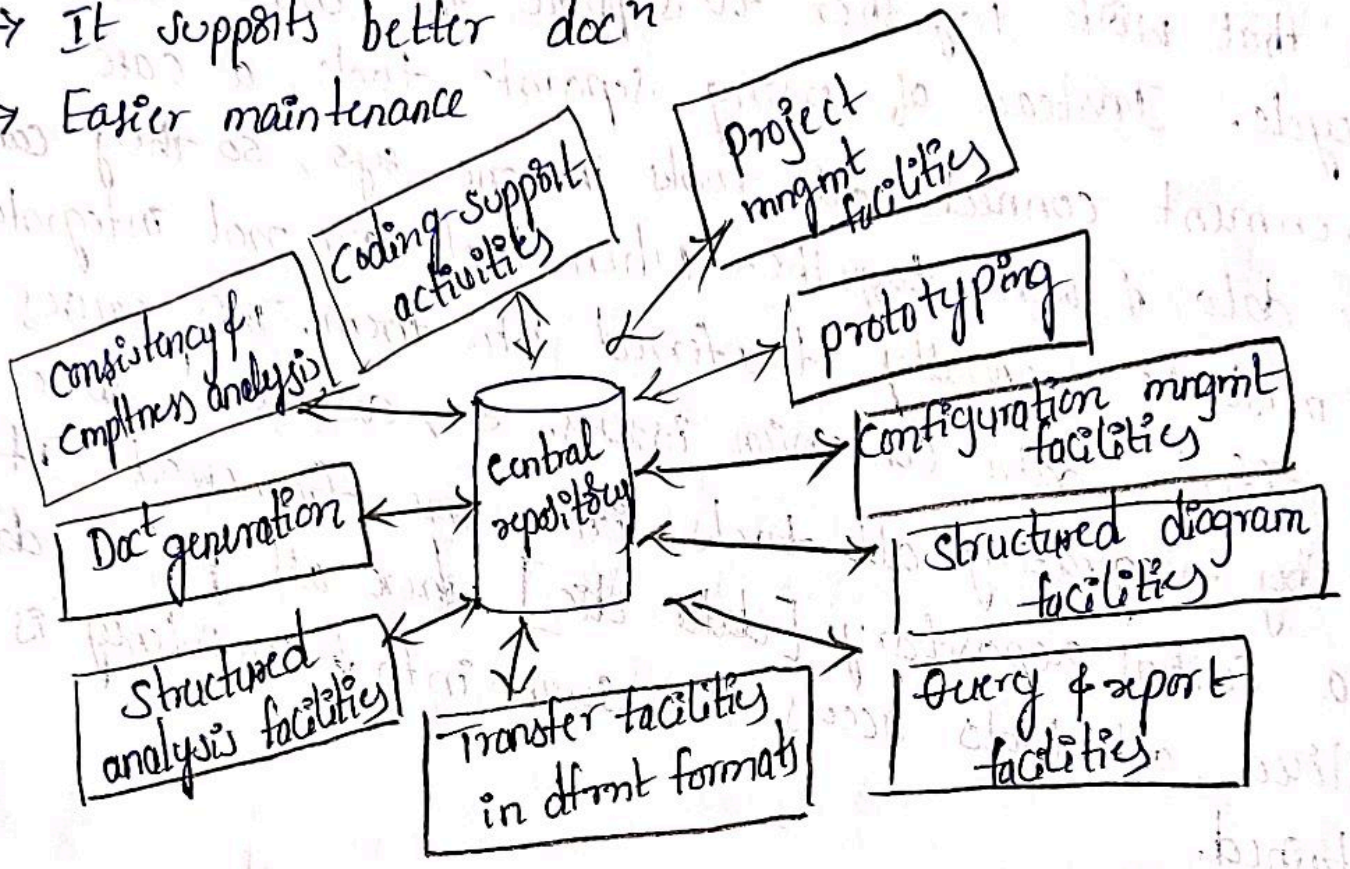
Features

→ It support all devt stages
→ Integration of multiple CASE tools
→ cmn data sharing through repository
→ It improves efficiency & consistency

CASE Env supports entire s/w devt. In contrast to a case env, a prmng env is an integrated collection of tools to support only the coding phase of s/w devt. It includes tools like text editors, compiler, Debugger & Linker
Ex Turbo c, visual basic, visual c++

Benefits

→ cost & effort reduction
→ Improved s/w qua
→ It supports better doc^{n}
→ Easier maintenance



(3) CASE Support in s/w life cycle

CASE tools help dev^{e} during all phases of the SDLC. They impove productivity, consistency & quality by supporting methods, checking errors, & maintaining proper doc^{n}.

prototyping support :- creating a sample or model of the s/w before final dev^{t}. The purpose of prototyping is, to understand user req^{s}, to demonstrate ideas & to test concepts before actual dev^{t}.

Features → It helps to define user interaction
→ It defines system flow
→ It stores & retrieve data.

A good prototyping tool should support the following imp capabilities → It creates GUI using editors
→ It connects with data dictionary
→ The user should be able to allow defining execution sequence of prototype. → It supports mock run.

Structured analysis & design :- This phase involves creating diagms & models to understand & design the sys. It helps draw diagrams like DFD & structure charts. It supports hierarchical design [top to detailed]

Features → It ensures consistency btn diffnt diagrams.
→ It checks for completeness.
→ It allows creating complex diagrams in multiple levels.
→ It provides easy navigation btn diffnt levels & diagrams.
→ The tool should prevent wrong actions.

Code generation :- CASE tools provides limited but useful support in coding. It reduces manual coding effort & maintains consistency.

Features → It generates module templates
→ It automatically creates structures, classes & records
→ It generates database tables.
→ It creates user interface code from prototype designs.

Test case generator :- This tool helps in testing the s/w. It helps to ensure s/w works correctly by generating test cases.

Features → It supports req testing & design testing
→ It automatically generates test cases.

(4) other characteristics of CASE tools

H/W & Environmental req^{n} :- case tools should work efficiently with existing h/w sys. They support multi user envts using client-server architecture & allow data sharing through a central data dictionary.

Doc^{n} support :- case tools help in creating well organized & up-to-date doc^{s} by combining text, diagrams, data & allow exporting to publishing formats.

project mngmt :- It should support collecting, storing & analysing info on the s/w projects progress such as the estimated task duration, scheduled & actual task start/end dates & results of the reviews.

External Interface :- It provides data exchange with other sys using standard formats & allow integration with external tools through open interfaces.

Review Engineering support :- It can convert existing source code into design diagrams & data structures, helping in understanding & improving old systems.

Data dictionary interface :- This interface allows users to view, update, analyze project data & generates reports

Tutorial & Help :- It provides tutorials & doc^{n} to help users understand & effectively use all features of the tool.

(5) Towards Second Generation case tools :- Second gen CASE tools are more advanced tools that provides better support for s/w devt methodologies & allows customization.

Methodology support :- Second gen case tools can directly support dfrnt devt methods. A CASE administrator can modify the tool according to the org's needs.

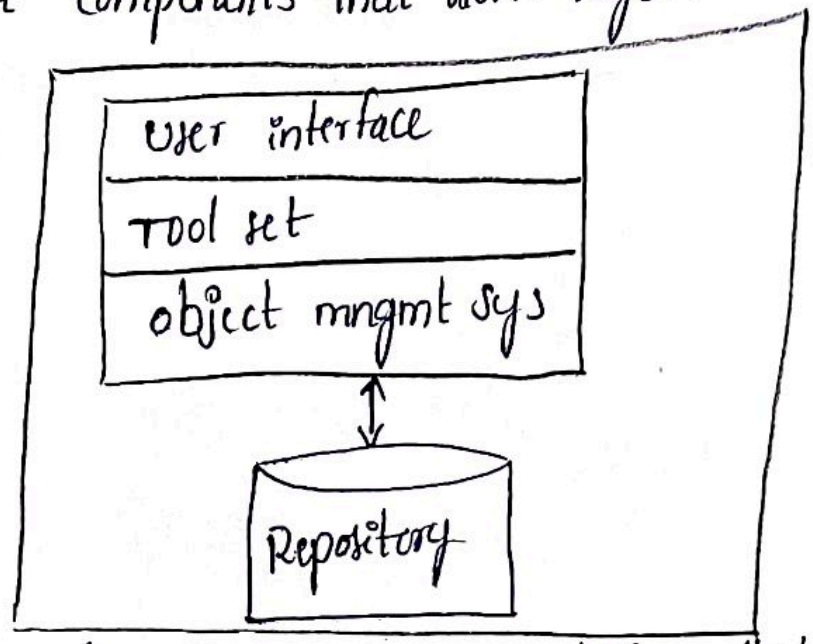
Intelligent diagramming support :- These tools helps in automatically creating neat & well structured diagrams, it makes sys design easier & clearer.

Integration with Implementation Enviironment :- These tools provides connection btn design & coding, so designs can be easily converted into actual programs.

Data dic standards :- They support standard data dic formats, allowing dfrnt tools to work together in one env.

customization support :- users can create their own objects, rules & connections to specific project needs. They can also define rules for checking consistency automatically.

(6) Architecture of a CASE Envt A CASE Envt consist of dfrnt components that work together to support s/w devt



User interface :- is a common platform that allows users to interact with dfrnt case tools easily & provides a consistent working envt.

Tool set :- is a collection of various case tools used for dfrnt activities like analysis, design, coding & testing.

object mngmt sys :- is a sys that manages & organizes all project data by mapping s/w entities into storage form.

Repository :- is a central storage loc where all project data such as design, code, & docs are stored & maintained.

Part - II

S/W Maintenance :- is the process of modifying & updating a s/w system after it is dest to keep it useful, adaptable and efficient in changing reqs. when ur mobile app gets an update

Need for s/w Maintenance is required due to

→ H/W changes
→ s/w envt changes [new os, platforms]
→ User demands for new features

Typs

(i) Corrective Maintenance :- is the process of fixing errors or failures found during s/w usage. app is not opening

(ii) Adaptive Maintenance :- is the process of modifying s/w to work in new envts, such as new h/w, os or interfaces. app updated to work on new android version

(iii) Perfective Maintenance :- is the process of enhancing s/w by adding new features or improving performance according to user reqs. improving speed

characteristics

Lehman's law

first law [continuing change]

A s/w sys must continuously change & evolve, ow it becomes less & useful over time. It not updated → becomes obs.

second law [Increasing complexity] more changes → messy code

The structure of s/w deteriorates & becomes more complex as more changes are made during maintenance.

Third law [Constant work Rate]

The rate of s/w dev't & maintenance remains approx constant throughout its lifetime.

problems in s/w Maintenance

Ad hoc maintenance s/w main is often done in an unplanned & unsystematic way; leading to inefficiency.

poor image of main main work has a low reputation, making it difficult to assign skilled engineers, even though it is complex work.

Legacy sys prbm is a s/w sys that is difficult to maintain due to poor design, outdated technology & lack of doc^n

S/W Reverse Engineering is the process of understanding a s/w sys by analyzing its code when proper documentation is not available. It means studying an existing prgm to know how it works & how it was designed. Many old s/w sys do not have proper doc's & difficult to understand. over time s/w becomes confusing due to many changes. So, reverse engineering helps to improve understanding, make maintenance easier & recreate missing doc's like design reqs.

Stages of Rev Eng

(i) Cosmetic changes Before understanding the sys, we improve the code appearance without changing the functionality. This includes reformatting the code, simplifying complex conditions, making code easier to read.

(ii) understanding the sys To analyze how the program works & to identify data flow & control flow

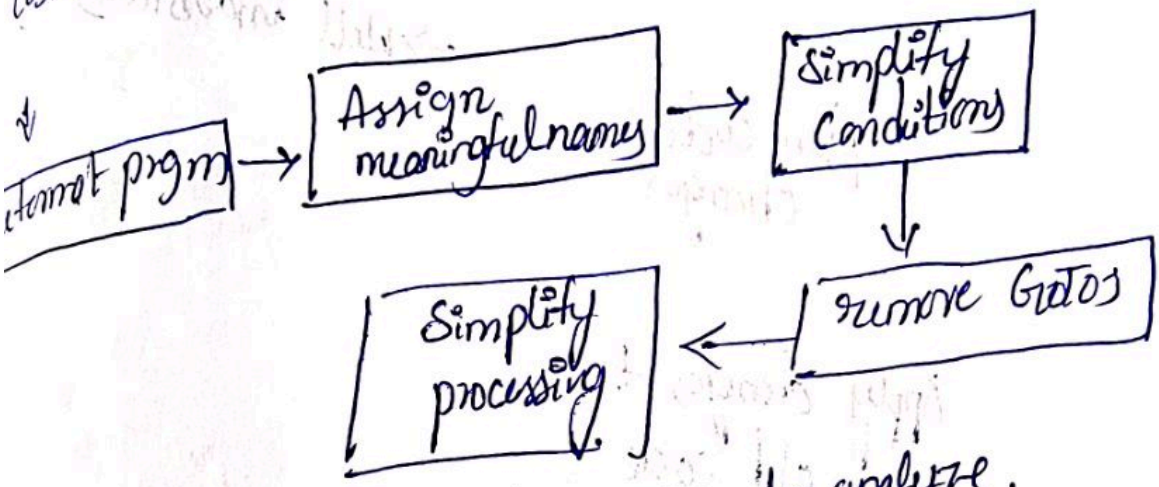
(iii) Extracting design & reqs To identify design architecture finally recreate design doc, reqs specification.

process model

Normal dev^t :- reqs → design → code

Rev Eng :- Speciti → design → reqs → code

cosmetic changes carried out before rev eng



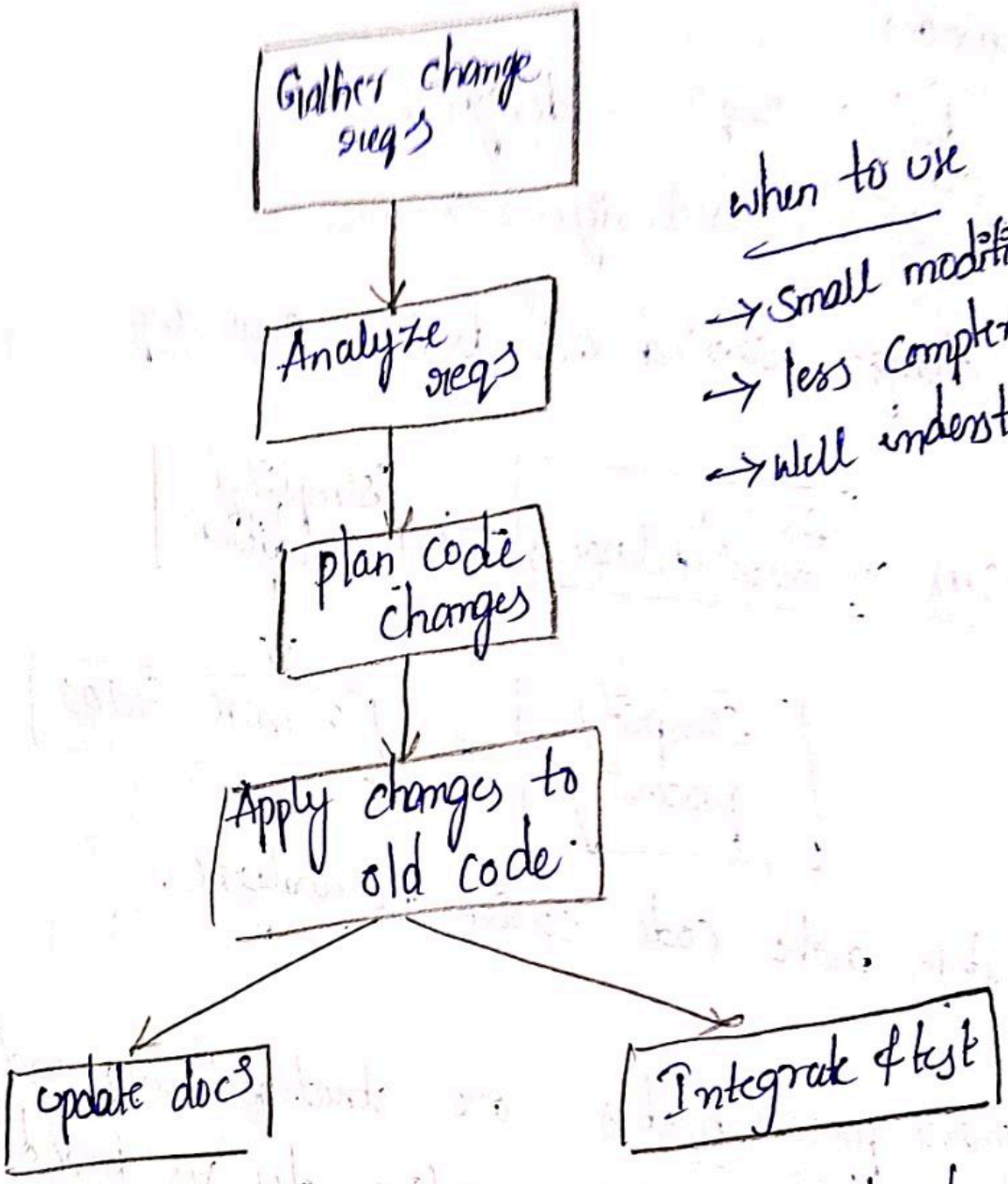
These steps make code easier to analyze.

S/w Maintenance process models are structured approaches used to modify, improve or update existing s/w sys based on changing reqs, errors or env^t changes. These models help decide how maintenance work should be carried out depending on the size & complexity of changes

Types

(i) Direct modification model [first]

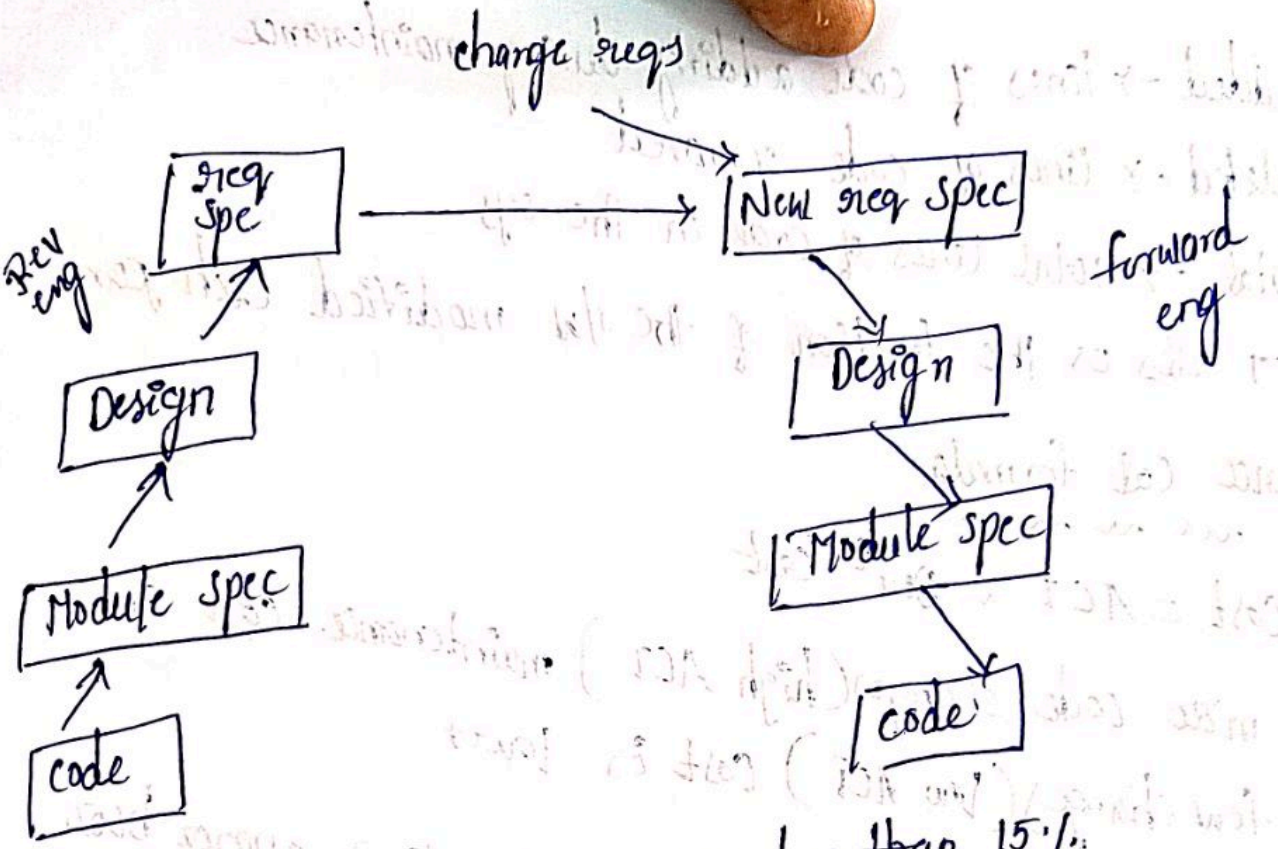
This model is used when small changes are needed. The existing code is directly modified later the docs are updated.



when to use
→ Small modifications
→ less complex sys
→ well understood sw

(ii) Re-engineering model This model is used when major changes are required. It involves reverse engin [understanding old sys] followed by forward eng [developing improved sys]

When to use → large rework
→ legacy systems
→ poor doc^{n} / design



process 1 → best when changes are less than 15%.
process 2 → better when changes are large / complex

Estimation of Maintenance cost s/w maintenance is a major part of s/w cost. It usually takes about 60% of the total life cycle cost. In some sys [like embedded sy] maintenance cost can be 2 to 4 times the dev^t cost.

Bochm's approach Barry bochm proposed a method to estimate maintenance cost using Annual change Traffic (ACT). It measures how much of the s/w changes in one year.

$$ACT = \frac{\text{KLOC added} + \text{KLOC deleted}}{\text{KLOC total}}$$

KLOC added → lines of code adding during maintenance
Kloc deleted → Lines of code removed
Kloc total → total lines of code in the sys
So, ACT tells us the fraction of the s/w modified each year.

Maintenance cost formula

Main cost = ACT x Dev^{t} cost

→ If more code changes (high ACT) maintenance cost inc.
→ If few changes (low ACT) cost is lower

Limitations of estimation models Main cost esti is approx bcoz it does not consider
→ Experience level of engineers
→ familiarity with the sys
→ s/w complexity
→ H/w reqs
→ tools & technologies used.